Contents

January 26, 2026

1 Data Structures

1.1 Disjoint Set Union (DSU)

```
/* DSU (Disjoint Set Union / Union-Find)
 * Mantem conjuntos disjuntos com uniao e busca eficiente.
 * Complexidade: O(a(N)) por operacao, onde a e a inversa de Ackermann.
 * Memoria: O(N)
 * Metodos:
 * - find(i): Retorna o representante do conjunto de i.
 * - join(i, j): Une os conjuntos de i e j, retorna true se uniu.
 * - same(i, j): Verifica se i e j estao no mesmo conjunto.
 * - size(i): Retorna o tamanho do conjunto de i.
 */

struct DSU {
    int n;
    vector<int> parent;
    vector<int> sz;
    int num_sets;

    DSU(int _n) : n(_n), parent(_n), sz(_n, 1), num_sets(_n) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]);
    }

    bool join(int i, int j) {
        i = find(i), j = find(j);
        if (i != j) {
            if (sz[i] < sz[j]) swap(i, j);
            parent[j] = i;
            sz[i] += sz[j];
            num_sets--;
            return true;
        }
        return false;
    }
    int size(int i) { return sz[find(i)]; }
    bool same(int i, int j) { return find(i) == find(j); }
};
```

1.2 Dynamic Fenwick Tree2d

```
/* Fenwick Tree 2D Sparse (Point Update, Rectangle Query)
 * Suporta coordenadas ate 10^9 usando compressao interna.
 * Complexidade: Build O(P log P), Update/Query O(log^2 P).
 * Memoria: O(P log P).
 * Requisitos:
 * - NODE deve ter: operator+=, operator-, operator+ e construtor default.
 * - Offline: Todos os pontos de interesse devem ser passados no build.
 *
 * Creditos: Adaptado de TFG (Thiago Ferreira Goncalves)
 * Fonte: github.com/tfg50/Competitive-Programming
 */

/* --- Exemplo de NODE (Soma com Inclusao-Exclusao) ---
struct Node {
    ll val;
    Node(long long v = 0) : val(v) {}

    void operator+=(const Node& other) {
        val += other.val;
    }

    Node operator-(const Node& other) const {
        return Node(val - other.val);
    }

    Node operator+(const Node& other) const {
        return Node(val + other.val);
    }
};
*/

template<typename NODE>
struct FenwickTree2DSparse {
    vector<int> ord;
    vector<vector<NODE>> bit;
    vector<vector<int>> coord;

    FenwickTree2DSparse(vector<pii> pts) {
        sort(pts.begin(), pts.end());
        for (auto [x, y] : pts) {
            if (ord.empty() || x != ord.back()) ord.push_back(x);
        }

        bit.resize(ord.size() + 1);
        coord.resize(ord.size() + 1);

        sort(pts.begin(), pts.end(), [](const pii& a, const pii& b) {
            return a.second < b.second;
        });

        for (auto [x, y] : pts) {
            for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
                if (coord[i].empty() || coord[i].back() != y)
                    coord[i].push_back(y);
            }
        }

        for (int i = 0; i < (int)bit.size(); i++) {
            bit[i].assign(coord[i].size() + 1, NODE());
        }
    }

    inline int get_x(int x) {
        return upper_bound(ord.begin(), ord.end(), x) - ord.begin();
    }
    inline int get_y(int i, int y) {
        return upper_bound(coord[i].begin(), coord[i].end(), y)
               - coord[i].begin();
    }

    void update(int x, int y, NODE v) {
        for (int i = get_x(x); i < (int)bit.size(); i += i & -i) {
            for (int j = get_y(i, y); j < (int)bit[i].size(); j += j & -j) {
                bit[i][j] += v;
            }
        }
    }

    NODE query(int x, int y) {
        NODE res;
        for (int i = get_x(x); i > 0; i -= i & -i) {
            for (int j = get_y(i, y); j > 0; j -= j & -j) {
                res += bit[i][j];
            }
        }
        return res;
    }

    NODE query(int x1, int y1, int x2, int y2) {
        return query(x2, y2) - query(x1 - 1, y2)
               - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
    }
};
```

1.3 Fenwick Tree

```
/* Fenwick Tree (Point Update, Prefix Query)
 * Realiza atualizacoes pontuais e consultas de prefixo.
 * Complexidade: O(log N) para update e query.
 * Memoria: O(N)
 * Requisitos:
 * - NODE deve ter: operator+=, operator- (para range query) e
 *   construtor default.
 */

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

template<typename NODE>
struct FenwickTree {
    int N;
    vector<NODE> bit;

    FenwickTree(int n) : N(n), bit(n + 1) {}

    void update(int idx, NODE v) {
        for (idx++; idx <= N; idx += idx & -idx) {
            bit[idx] += v;
        }
    }

    NODE query(int idx) { // [0, idx]
        NODE res;
        for (idx++; idx > 0; idx -= idx & -idx) {
            res += bit[idx];
        }
        return res;
    }

    NODE query(int l, int r) { // [l, r] (precisa ser reversivel)
```

```
        if (l > r) return NODE();
        return query(r) - query(l - 1);
    }
};
```

1.4 Fenwick Tree2d

```
/* Fenwick Tree 2D (Point Update, Rectangle Query)
 * Realiza atualizacoes pontuais e consultas de prefixo em matrizes.
 * Complexidade: O(log N * log M) para update e query.
 * Memoria: O(N * M)
 * Requisitos:
 * - NODE deve ter: operator+=, operator- e construtor default.
 */

/* --- Exemplo de NODE (Soma) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    void operator+=(const Node& other) { val += other.val; }
    Node operator-(const Node& other) const { return Node(val - other.val); }
};
*/

template<typename NODE>
struct FenwickTree2D {
    int N, M;
    vector<vector<NODE>> bit;

    FenwickTree2D(int n, int m) : N(n), M(m), bit(n + 1, vector<NODE>(m + 1)) {}

    void update(int r, int c, NODE v) { // point update
        for (int i = r + 1; i <= N; i += i & -i) {
            for (int j = c + 1; j <= M; j += j & -j) {
                bit[i][j] += v;
            }
        }
    }

    NODE query(int r, int c) { // [(0,0), (r,c)]
        NODE res;
        for (int i = r + 1; i > 0; i -= i & -i) {
            for (int j = c + 1; j > 0; j -= j & -j) {
                res += bit[i][j];
            }
        }
        return res;
    }

    NODE query(int r1, int c1, int r2, int c2) { // [(r1,c1), (r2,c2)]
        if (r1 > r2 || c1 > c2) return NODE();
        return query(r2, c2) - query(r1 - 1, c2)
               - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
    }
};
```

1.5 Dual Segment Tree

```
/* Dual Segment Tree (Range Update, Point Query)
 * Realiza atualizacoes em range e consultas pontuais.
 * Complexidade: O(log N) para update e get.
 * Memoria: O(4*N)
 * Requisitos:
 * - TAG deve ter: compose(TAG), apply(T&) e construtor identidade.
 * - T e o tipo do dado nas folhas.
 * Exemplo de uso: DualSegTree<ll, MyTag> st(v);
 */

/* --- Exemplo de TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void compose(const Tag& t) {
        mul = (mul * t.mul) % MOD;
        add = (add * t.mul + t.add) % MOD;
    }
    void apply(ll& val) { val = (val * mul + add) % MOD; }
};
*/

template<typename T, typename TAG>
struct DualSegTree {
    int N;
    vector<TAG> lazy;
    vector<T> leaves;

    DualSegTree(int n) : N(n), lazy(4 * n), leaves(n) {}
    DualSegTree(const vector<T>& v)
        : N(v.size()), lazy(4 * v.size()), leaves(v) {}

    void push(int no, int l, int r) {
        int e = no << 1;
        int d = e | 1;
        lazy[e].compose(lazy[no]);
        lazy[d].compose(lazy[no]);
        lazy[no] = TAG();
    }

    void update(int no, int l, int r, int a, int b, const TAG& v) {
        if (b < l || r < a) return;
        if (a <= l && r <= b) {
            lazy[no].compose(v);
            return;
        }
        push(no, l, r);
        int m = (l + r) >> 1;
        update(no << 1, l, m, a, b, v);
        update((no << 1) | 1, m + 1, r, a, b, v);
    }

    T get(int no, int l, int r, int idx) {
        if (l == r) {
            T res = leaves[idx];
            lazy[no].apply(res);
            return res;
        }
        push(no, l, r);
        int m = (l + r) >> 1;
        if (idx <= m) return get(no << 1, l, m, idx);
        else return get((no << 1) | 1, m + 1, r, idx);
    }

    void update(int l, int r, const TAG& t) { update(1, 0, N - 1, l, r, t); }
    T get(int idx) { return get(1, 0, N - 1, idx); }
};
```

1.6 Dynamic Segment Tree

```
/* Dynamic Segment Tree with Lazy Propagation (Range Update, Range Query)
 * Utilizada para intervalos gigantes (ex: 0 a 10^18).
 * Complexidade: O(log N) por operacao.
 * Memoria: O(Q log N) onde Q e o numero de operacoes.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&),
 *   apply(const TAG&, ll, ll) e construtor identidade.
 * - TAG deve ter: apply(const TAG&), campo 'has' (bool) e
 *   construtor identidade.
 */

/* --- Exemplo de NODE e TAG ---
struct Tag {
    ll add = 0;
    bool has = false;
    void apply(const Tag& t) {
        add += t.add;
        has = true;
    }
};

struct Node {
    ll val = 0;
    static Node merge(const Node& l, const Node& r) {
        return {l.val + r.val};
    }
    void apply(const Tag& t, ll l, ll r) {
        val += t.add * (r - l + 1);
    }
};
*/

template<typename NODE, typename TAG>
struct DynamicSegTree {
    struct InternalNode {
        NODE node;
        TAG tag;
        int l, r;
        InternalNode() : node(NODE()), tag(TAG()), l(-1), r(-1) {}

        void apply(const TAG& t, ll l, ll r) {
            node.apply(t, l, r);
            tag.apply(t);
        }
    };

    ll L, R;
    vector<InternalNode> seg;

    DynamicSegTree(ll l, ll r) : L(l), R(r) {
        seg.emplace_back();
    }

    void push(int v, ll l, ll r) {
        if (!seg[v].tag.has) return;
        if (l == r) return seg[v].tag = TAG();

        if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
        if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }

        ll mid = l + (r - l) / 2;
        seg[seg[v].l].apply(seg[v].tag, l, mid);
        seg[seg[v].r].apply(seg[v].tag, mid+1, r);

        seg[v].tag = TAG();
    }

    void update(int v, ll l, ll r, ll ql, ll qr, const TAG& t) {
        if (ql <= l && r <= qr) return seg[v].apply(t, l, r);

        push(v, l, r);
        ll mid = l + (r - l) / 2;
        if (ql <= mid) {
            if (seg[v].l == -1) { seg[v].l = seg.size(); seg.emplace_back(); }
            update(seg[v].l, l, mid, ql, qr, t);
        }
        if (qr > mid) {
            if (seg[v].r == -1) { seg[v].r = seg.size(); seg.emplace_back(); }
            update(seg[v].r, mid + 1, r, ql, qr, t);
        }

        NODE left = (seg[v].l != -1) ? seg[seg[v].l].node : NODE();
        NODE rght = (seg[v].r != -1) ? seg[seg[v].r].node : NODE();
        seg[v].node = NODE::merge(left, rght);
    }

    NODE query(int v, ll l, ll r, ll ql, ll qr) {
        if (v == -1 || ql > r || qr < l) return NODE();
        if (ql <= l && r <= qr) return seg[v].node;
```

```
        push(v, l, r);
        ll mid = l + (r - l) / 2;
        return NODE::merge(query(seg[v].l, l, mid, ql, qr),
                           query(seg[v].r, mid + 1, r, ql, qr));
    }

    void update(ll ql, ll qr, const TAG& t) { update(0, L, R, ql, qr, t); }
    NODE query(ll ql, ll qr) { return query(0, L, R, ql, qr); }
};
```

1.7 Lazy Segment Tree

```
/* Lazy Segment Tree (Range Update, Range Query)
 * Realiza atualizacoes e consultas em range.
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(TAG, L, R) e
 *   construtor identidade.
 * - TAG deve ter: compose(TAG) e construtor identidade.
 */

/* --- Exemplo de NODE e TAG (Soma e Multiplicacao com Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void inline compose(const Tag& t) {
        add = (add * t.mul + t.add) % MOD;
        mul = (mul * t.mul) % MOD;
    }
};

struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node((l.val + r.val) % MOD);
    }
    void inline apply(const Tag& t, int l, int r) {
        val = (val * t.mul + t.add * (r - l + 1)) % MOD;
    }
};
*/

template<typename NODE, typename TAG>
struct LazySegmentTree {
    int N;
    vector<NODE> seg;
    vector<TAG> lazy;

    LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n) {}

    LazySegmentTree(const vector<int>& v)
        : N(v.size()), seg(4 * v.size()), lazy(4 * v.size()) {
        build(1, 0, N - 1, v);
    }

    void build(int no, int l, int r, const vector<int>& v) {
        if (l == r) {
            seg[no] = NODE(v[l]);
            return;
        }
        int m = (l + r) >> 1;
        build(no << 1, l, m, v);
        build((no << 1) | 1, m + 1, r, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    void push(int no, int l, int r) {
        int m = (l + r) >> 1;
        int e = no << 1, d = e | 1;

        seg[e].apply(lazy[no], l, m);
        lazy[e].compose(lazy[no]);

        seg[d].apply(lazy[no], m + 1, r);
        lazy[d].compose(lazy[no]);

        lazy[no] = TAG();
    }

    void update(int no, int l, int r, int a, int b, const TAG& v) {
        if (b < l || r < a) return;
        if (a <= l && r <= b) {
            seg[no].apply(v, l, r);
            lazy[no].compose(v);
            return;
        }
        push(no, l, r);
        int m = (l + r) >> 1;
        update(no << 1, l, m, a, b, v);
        update((no << 1) | 1, m + 1, r, a, b, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    NODE query(int no, int l, int r, int a, int b) {
        if (b < l || r < a) return NODE();
        if (a <= l && r <= b) return seg[no];
        push(no, l, r);
        int m = (l + r) >> 1;
        return NODE::merge(query(no << 1, l, m, a, b),
                           query((no << 1) | 1, m + 1, r, a, b));
    }

    void update(int l, int r, const TAG& v) { update(1, 0, N - 1, l, r, v); }
    NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
};
```

1.8 Persistent Segment Tree

```
/* Persistent Segment Tree (Point Update, Range Query)
 * Permite consultas em versoes anteriores da arvore.
 * Complexidade: O(log N) por update e query.
 * Memoria: O(Q log N) onde Q e o numero de updates.
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e
 *   construtor identidade.
 */

/* --- Exemplo de NODE (Soma) ---
struct Node {
    int val;
    Node(int v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

template<typename NODE>
struct PersistentSegmentTree {
    struct InternalNode {
        NODE data;
        int l, r;
        InternalNode() : data(NODE()), l(0), r(0) {}
    };

    int N;
    vector<InternalNode> st;
    vector<int> roots;

    PersistentSegmentTree(int n) : N(n) {
        st.emplace_back(); // No 0 e o no nulo
        roots.push_back(build(0, N - 1));
    }

    int build(int L, int R) {
        int node = st.size();
        st.emplace_back();
        if (L == R) return node;
        int mid = (L + R) / 2;
        st[node].l = build(L, mid);
        st[node].r = build(mid + 1, R);
        st[node].data = NODE::merge(st[st[node].l].data, st[st[node].r].data);
        return node;
    }

    int update(int prev_root, int L, int R, int idx, NODE v) {
        int node = st.size();
        st.push_back(st[prev_root]);
        if (L == R) {
            st[node].data = NODE::merge(st[node].data, v);
            return node;
        }
        int mid = (L + R) / 2;
        if (idx <= mid) st[node].l = update(st[prev_root].l, L, mid, idx, v);
        else st[node].r = update(st[prev_root].r, mid + 1, R, idx, v);

        st[node].data = NODE::merge(st[st[node].l].data, st[st[node].r].data);
        return node;
    }

    NODE query(int node, int L, int R, int i, int j) {
        if (i > R || j < L) return NODE();
        if (i <= L && R <= j) return st[node].data;
        int mid = (L + R) / 2;
        return NODE::merge(query(st[node].l, L, mid, i, j),
                           query(st[node].r, mid + 1, R, i, j));
    }

    void update(int idx, NODE v) {
        roots.push_back(update(roots.back(), 0, N - 1, idx, v));
    }

    NODE query(int version, int l, int r) {
        return query(roots[version], 0, N - 1, l, r);
    }
};
```

1.9 Segment Tree

```
/* Segment Tree (Point Update, Range Query)
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(V), e construtor identidade.
 */

/* --- Exemplo de NODE (Soma com Update de Substituicao) ---
struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}

    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }

    // Para Point Set: val = v;
    // Para Point Add: val += v;
    inline void apply(int v) {
        val = v;
    }
};
*/

template<typename NODE>
struct SegTree {
```

```
    int N;
    vector<NODE> seg;

    SegTree(int n) : N(n), seg(4 * n) {}

    SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
        build(1, 0, N - 1, v);
    }

    void build(int no, int l, int r, const vector<int>& v) {
        if (l == r) {
            seg[no] = NODE(v[l]);
            return;
        }
        int m = (l + r) >> 1;
        build(no << 1, l, m, v);
        build((no << 1) | 1, m + 1, r, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    void update(int no, int l, int r, int idx, int val) {
        if (l == r) {
            seg[no].apply(val);
            return;
        }
        int m = (l + r) >> 1;
        if (idx <= m) update(no << 1, l, m, idx, val);
        else update((no << 1) | 1, m + 1, r, idx, val);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    NODE query(int no, int l, int r, int a, int b) {
        if (b < l || r < a) return NODE();
        if (a <= l && r <= b) return seg[no];
        int m = (l + r) >> 1;
        return NODE::merge(query(no << 1, l, m, a, b),
                           query((no << 1) | 1, m + 1, r, a, b));
    }

    void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
    NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
};
```

1.10 Disjoint Sparse Table

```
/* Disjoint Sparse Table (Static Range Query)
 * Realiza consultas em range em O(1) para qualquer operacao associativa.
 * Complexidade: O(N log N) no build, O(1) na query.
 * Memoria: O(N log N)
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
 * - Operacao deve ser ASSOCIATIVA: f(f(a,b),c) = f(a,f(b,c)).
 */

/* --- Exemplo de NODE (Soma em Range) ---
struct Node {
    ll val;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }
};
*/

template<typename NODE>
struct DisjointSparseTable {
    int N, K;
    vector<vector<NODE>> st;

    template<typename T>
    DisjointSparseTable(const vector<T>& v) : N(v.size()) {
        if (N == 0) return;
        K = __lg(2 * N - 1);
        st.assign(K, vector<NODE>(1 << K));

        for (int i = 0; i < N; i++) st[0][i] = NODE(v[i]);

        for (int j = 1; j < K; j++) {
            int len = 1 << j;
            for (int m = len; m <= (1 << K); m += 2 * len) {
                if (m < N) st[j][m] = st[0][m];
                for (int i = m + 1; i < min(N, m + len); i++)
                    st[j][i] = NODE::merge(st[j][i - 1], st[0][i]);

                if (m - 1 < N) st[j][m - 1] = st[0][m - 1];
                for (int i = m - 2; i >= m - len; i--)
                    st[j][i] = NODE::merge(st[0][i], st[j][i + 1]);
            }
        }
    }

    inline NODE query(int l, int r) {
        if (l == r) return st[0][l];
        int j = __lg(l ^ r);
        return NODE::merge(st[j][l], st[j][r]);
    }
};
```

1.11 Sparse Table

```
/* Sparse Table (Static RMQ)
 * Realiza consultas em range em tempo constante.
 * Complexidade: O(N log N) no build, O(1) na query.
 * Memoria: O(N log N)
 * Requisitos:
 * - NODE deve ter: static merge(const NODE&, const NODE&) e construtor.
 * - A operacao DEVE ser idempotente: f(a, a) = a.
 */

/* --- Exemplo de NODE (Indice do Minimo) ---
struct Node {
    int val, pos;
    Node(int v = 2e9, int p = -1) : val(v), pos(p) {}

    static inline Node merge(const Node& l, const Node& r) {
        if (l.val <= r.val) return l;
        return r;
    }
};
*/

template<typename NODE>
struct SparseTable {
    int N, K;
    vector<vector<NODE>> st;
    vector<int> lg;

    template<typename T>
    SparseTable(const vector<T>& v) : N(v.size()) {
        K = (N > 0) ? __lg(N) : 0;
        st.assign(K + 1, vector<NODE>(N));
        lg.assign(N + 1, 0);
        for (int i = 2; i <= N; i++) lg[i] = lg[i / 2] + 1;
        for (int i = 0; i < N; i++) st[0][i] = NODE(v[i], i);
        for (int j = 1; j <= K; j++)
            for (int i = 0; i + (1 << j) <= N; i++)
                st[j][i] = NODE::merge(st[j - 1][i],
                                       st[j - 1][i + (1 << (j - 1))]);
    }

    NODE query(int l, int r) {
        int j = lg[r - l + 1];
        return NODE::merge(st[j][l], st[j][r - (1 << j) + 1]);
    }
};
```

2 Graphs

2.1 Lowest Common Ancestor (LCA)

```
#include "../data-structures/sparse-table/sparse-table.hpp"

/* LCA (Lowest Common Ancestor) via RMQ
 * Encontra o ancestral comum mais proximo de dois vertices em uma arvore.
 * Complexidade: O(N log N) no build, O(1) por query.
 * Memoria: O(N log N)
 * Requisitos:
 * - Grafo deve ser uma arvore (conexo e aciclico).
 * - Dependencia: sparse-table/sparse-table.hpp
 * Metodos:
 * - lca(u, v): Retorna o LCA de u e v.
 * - dist(u, v): Retorna a distancia entre u e v.
 * - is_ancestor(u, v): Verifica se u e ancestral de v.
 * - parent(u): Retorna o pai de u.
 */

struct LCANode {
    int dep, id;
    LCANode(int d = 1e9, int pos = -1) : dep(d), id(pos) {}

    static inline LCANode merge(const LCANode& l, const LCANode& r) {
        return l.dep < r.dep ? l : r;
    }
};

struct LCA {
    int N;
    vector<int> first, tour, d, p;
    SparseTable<LCANode> st;

    LCA(int n, int root, const vector<vector<int>>& adj)
        : N(n), first(n), d(n), p(n, -1), st(vector<LCANode>()) {

        vector<int> tour_depths;
        tour.reserve(2 * n);
        tour_depths.reserve(2 * n);

        auto dfs = [&](auto self, int u, int parent_id, int dep) -> void {
            p[u] = parent_id;
            d[u] = dep;
            first[u] = tour.size();
            tour.push_back(u);
            tour_depths.push_back(dep);
            for (int v : adj[u]) {
                if (v == parent_id) continue;
                self(self, v, u, dep + 1);
                tour.push_back(u);
                tour_depths.push_back(dep);
            }
        };

        dfs(dfs, root, -1, 0);
        st = SparseTable<LCANode>(tour_depths);
    }

    int parent(int u) {
        return p[u];
    }

    int lca(int u, int v) {
        int l = first[u], r = first[v];
        if (l > r) swap(l, r);
        return tour[st.query(l, r).id];
    }
```

```
    int dist(int u, int v) {
        return d[u] + d[v] - 2 * d[lca(u, v)];
    }

    bool is_ancestor(int u, int v) {
        return lca(u, v) == u;
    }
};
```

2.2 Hld Commutative Lazy

```
#include "../../data-structures/segment-tree/lazy-segment-tree.hpp"

/* HLD Comutativo com Lazy Propagation
 * Para operacoes onde A + B = B + A.
 * Permite updates e queries em caminhos e subarvores.
 * Requisito: data-structures/segment-tree/lazy-segment-tree.hpp
 */

template<typename NODE, typename TAG>
struct HLD {
    int n, t;
    vector<int> p, sz, d, head, pos;
    LazySegmentTree<NODE, TAG> st;

    HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
        : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

        vector<vector<int>> g = adj;
        d[root] = 0, p[root] = root;

        auto dfs_sz = [&](auto self, int u) -> void {
            sz[u] = 1;
            int best_v = -1, max_sz = -1;
            for (auto &v : g[u]) {
                if (v == p[u]) continue;
                d[v] = d[u] + 1; p[v] = u;
                self(self, v);
                sz[u] += sz[v];
                if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<NODE> base(n);
        for(int i = 0; i < n; i++)
            base[pos[i]] = vals[i];
        st = LazySegmentTree<NODE, TAG>(base);
    }

    void update_path(int u, int v, const TAG& tag) {
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) swap(u, v);
            st.update(pos[head[v]], pos[v], tag);
            v = p[head[v]];
        }
        if (d[u] > d[v]) swap(u, v);
        st.update(pos[u], pos[v], tag);
    }

    NODE query_path(int u, int v) {
        NODE res;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) swap(u, v);
            res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
            v = p[head[v]];
        }
        if (d[u] > d[v]) swap(u, v);
        return NODE::merge(res, st.query(pos[u], pos[v]));
    }

    void update_subtree(int u, const TAG& tag) {
        st.update(pos[u], pos[u] + sz[u] - 1, tag);
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1);
    }
};
```

2.3 Hld Commutative

```
#include "../../data-structures/segment-tree/segment-tree.hpp"

/* HLD Comutativo (Update Pontual)
 * Para operacoes onde A + B = B + A.
 * Requisito: data-structures/segment-tree/segment-tree.hpp
 */

template<typename NODE>
struct HLD {
    int n, t;
    vector<int> p, sz, d, head, pos;
    SegTree<NODE> st;

    HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
        : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

        vector<vector<int>> g = adj;
        d[root] = 0, p[root] = root;

        auto dfs_sz = [&](auto self, int u) -> void {
            sz[u] = 1;
            int best_v = -1, max_sz = -1;
            for (auto &v : g[u]) {
                if (v == p[u]) continue;
                d[v] = d[u] + 1; p[v] = u;
                self(self, v);
                sz[u] += sz[v];
                if (sz[v] > max_sz) {
                    max_sz = sz[v];
                    best_v = v;
                }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<NODE> base(n);
        for(int i = 0; i < n; i++)
            base[pos[i]] = vals[i];
        st = SegTree<NODE>(base);
    }

    void update(int u, const NODE& val) {
        st.update(pos[u], val);
    }

    NODE query_path(int u, int v) {
        NODE res;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) swap(u, v);
            res = NODE::merge(res, st.query(pos[head[v]], pos[v]));
            v = p[head[v]];
        }
        if (d[u] > d[v]) swap(u, v);
        return NODE::merge(res, st.query(pos[u], pos[v]));
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1);
    }
};
```

2.4 Hld Non Commutative Lazy

```
#include "../../data-structures/segment-tree/lazy-segment-tree.hpp"

/* HLD Nao-Comutativo com Lazy Propagation
 * Para operacoes onde A + B != B + A (ex: Matrizes, Funcoes Lineares).
 * Requisito: DoubleNode que guarda versoes 'normal' e 'invertida' do merge.
 */

template<typename NODE>
struct DoubleNode {
    NODE down, up;

    DoubleNode() : down(NODE()), up(NODE()) {}
    DoubleNode(const NODE& n) : down(n), up(n) {}
    DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

    static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
        return {
            NODE::merge(l.down, r.down),
            NODE::merge(r.up, l.up)
        };
    }

    template<typename TAG>
    void apply(const TAG& t, int l, int r) {
        down.apply(t, l, r);
        up.apply(t, l, r);
    }
};
```

```cpp
template<typename NODE, typename TAG>
struct HLD {
    int n, t;
    vector<int> p, sz, d, head, pos;
    LazySegmentTree<DoubleNode<NODE>, TAG> st;

    HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
        : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), st(n) {

        vector<vector<int>> g = adj;
        d[root] = 0, p[root] = root;

        auto dfs_sz = [&](auto self, int u) -> void {
            sz[u] = 1;
            int best_v = -1, max_sz = -1;
            for (auto &v : g[u]) {
                if (v == p[u]) continue;
                d[v] = d[u] + 1; p[v] = u;
                self(self, v);
                sz[u] += sz[v];
                if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<DoubleNode<NODE>> base(n);
        for(int i = 0; i < n; i++) base[pos[i]] = DoubleNode<NODE>(vals[i]);
        st = LazySegmentTree<DoubleNode<NODE>, TAG>(base);
    }

    void update_path(int u, int v, const TAG& tag) {
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) {
                st.update(pos[head[u]], pos[u], tag);
                u = p[head[u]];
            } else {
                st.update(pos[head[v]], pos[v], tag);
                v = p[head[v]];
            }
        }
        if (d[u] > d[v]) st.update(pos[v], pos[u], tag);
        else st.update(pos[u], pos[v], tag);
    }

    NODE query_path(int u, int v) {
        NODE L, R;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) {
                L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
                u = p[head[u]];
            } else {
                R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);
                v = p[head[v]];
            }
        }
        if (d[u] > d[v]) {
            L = NODE::merge(L, st.query(pos[v], pos[u]).up);
        } else {
            R = NODE::merge(st.query(pos[u], pos[v]).down, R);
        }
        return NODE::merge(L, R);
    }

    void update_subtree(int u, const TAG& tag) {
        st.update(pos[u], pos[u] + sz[u] - 1, tag);
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1).down;
    }
};
```

2.5 Hld Non Commutative

```cpp
#include "../../data-structures/segment-tree/segment-tree.hpp"

/* HLD Nao-Comutativo (Update Pontual)
 * Para operacoes onde A * B != B * A.
 * Requisito: DoubleNode para gerenciar merges em ambas as direcoes.
 */

template<typename NODE>
struct DoubleNode {
    NODE down, up;
    DoubleNode() : down(NODE()), up(NODE()) {}
    DoubleNode(const NODE& n) : down(n), up(n) {}
    DoubleNode(const NODE& d, const NODE& u) : down(d), up(u) {}

    static inline DoubleNode merge(const DoubleNode& l, const DoubleNode& r) {
        return {
            NODE::merge(l.down, r.down),
            NODE::merge(r.up, l.up)
        };
    }
};

template<typename NODE>
struct HLD {
    int n, t;
    vector<int> p, sz, d, head, pos, tour;
    SegTree<DoubleNode<NODE>> st;

    HLD(const vector<vector<int>>& adj, const vector<NODE>& vals, int root = 0)
     : n(adj.size()), t(0), p(n), sz(n), d(n), head(n), pos(n), tour(n), st(n) {

        vector<vector<int>> g = adj;
        d[root] = 0, p[root] = root;

        auto dfs_sz = [&](auto self, int u) -> void {
            sz[u] = 1;
            int best_v = -1, max_sz = -1;
            for (auto &v : g[u]) {
                if (v == p[u]) continue;
                d[v] = d[u] + 1; p[v] = u;
                self(self, v);
                sz[u] += sz[v];
                if (sz[v] > max_sz) { max_sz = sz[v]; best_v = v; }
            }
            if (best_v != -1) {
                for (int i = 0; i < (int)g[u].size(); i++) {
                    if (g[u][i] == best_v && i != 0) {
                        swap(g[u][0], g[u][i]);
                        break;
                    }
                }
            }
        };

        auto dfs_hld = [&](auto self, int u) -> void {
            pos[u] = t++;
            tour[pos[u]] = u;
            for (int i = 0; i < (int)g[u].size(); i++) {
                int v = g[u][i];
                if (v == p[u]) continue;
                head[v] = (i == 0 ? head[u] : v);
                self(self, v);
            }
        };

        dfs_sz(dfs_sz, root);
        head[root] = root;
        dfs_hld(dfs_hld, root);

        vector<DoubleNode<NODE>> base(n);
        for(int i = 0; i < n; i++)
            base[pos[i]] = DoubleNode<NODE>(vals[i]);
        st = SegTree<DoubleNode<NODE>>(base);
    }

    void update(int u, const NODE& val) {
        st.update(pos[u], DoubleNode<NODE>(val));
    }

    NODE query_path(int u, int v) {
        NODE L, R;
        while (head[u] != head[v]) {
            if (d[head[u]] > d[head[v]]) {
                L = NODE::merge(L, st.query(pos[head[u]], pos[u]).up);
                u = p[head[u]];
            } else {
                R = NODE::merge(st.query(pos[head[v]], pos[v]).down, R);
                v = p[head[v]];
            }
        }
        if (d[u] > d[v]) L = NODE::merge(L, st.query(pos[v], pos[u]).up);
        else R = NODE::merge(st.query(pos[u], pos[v]).down, R);

        return NODE::merge(L, R);
    }

    NODE query_subtree(int u) {
        return st.query(pos[u], pos[u] + sz[u] - 1).down;
    }
};
```

3 Extra

3.1 Hash Function

```cpp
/* Lib Hash
 * Gera hash de trechos de codigo.
 * Ignora comentarios e espacos em branco.
 * O hash de uma linha com } e o hash de todo o codigo desde a { que a abriu.
 * Uso:
 *   g++ hash.cpp -o hash
 *   ./hash < code.cpp
 */

string getHash(string s){
  ofstream("z.cpp") << s;
  system("g++ -E -P -dD -fpreprocessed ./z.cpp | tr -d '[:space:]' | md5sum > sh");
  ifstream("sh") >> s;
  return s.substr(0, 3);
}
```

```
E8D int main(){
973   string l, t;
C3E   stack<string> st({""});
C61   while(getline(cin, l)){
54F         t = l;
242     for(auto c : l)
A53       if(c == '{') st.push(""); else
733       if(c == '}') t = st.top()+l, st.pop();
1B9     cout << getHash(t) + " " + l << endl;
DB4     st.top() += t;
C14   }
692 }
```
